Lifted Gaussian Site Layers for In-Context Sensor Fusion

Anonymous Author(s)
Affiliation
Address
email

Abstract

Low-cost sensor deployments must estimate a target from whichever sensors work, after a short calibration, while temperature, humidity and drift perturb several sensors at once. We cast this as in-context probabilistic prediction that must operate under different missingness patterns and numbers of sensors. For linear-Gaussian sensors with shared noise, we show that mask-independent scalar Gaussian experts cannot in general reproduce every pattern's posterior. We propose *lifted cavity networks*: each observed sensor contributes a rank-one Gaussian site over the target and a low-dimensional shared nuisance, exact for factor-structured noise under every pattern once the nuisance dimension reaches the noise rank; necessity holds for static linear-Gaussian sites under a generic-rank condition. The sites start from a factor analysis of the incomplete calibration rows and are refined by a learned, cavity-conditioned re-linearization. The same support-calibrated sites also serve as a learned transformer output layer. On synthetic tasks with exact oracle and sign-conditioned HMC references, an 8k-parameter lifted cavity network closes 58% of the gap between the closed form and the HMC reference with two sensors missing, a pattern unseen in training, against 34% for a 25-times-larger TabPFN-v2-style transformer trained on the same tasks, and it transfers from five sensors to sixteen. After fine-tuning on real sensor networks it improves on the closed forms, but the transformer is better still. As the transformer's output layer, lifted sites beat the plain transformer on fresh synthetic tasks (by 0.035 nats with five sensors, 0.150 with sixteen) and recover most of the real-data gap on a new target, although non-inferiority to the fine-tuned transformer is not established. The three reported protocols preceded their test panels. A fourth GPU study is withheld after a provenance audit failed.

1 Introduction

Low-cost sensors now carry much of the world's environmental, industrial and clinical measurement. A typical deployment co-locates a few cheap sensors with a reference instrument for a short calibration period, and from then on predicts the reference quantity from the sensors alone [39]. Three features make this hard. First, every deployment is different: gains, cross-sensitivities and noise depend on the unit, its placement and its age, and the calibration record is short. Second, sensor noise is *shared*: temperature, humidity, local emissions and drift move several sensors at once, so sensors are not conditionally independent given the target. Third, sensors fail, are serviced or are added, so at prediction time an arbitrary subset is available, often one never seen together during calibration.

We study this as *in-context* probabilistic prediction. A network receives a deployment's labeled calibration rows (the support), with whatever entries are missing, and a query made of an arbitrary subset of sensor readings, and it returns a predictive distribution for the target. Trained once across many

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

deployments, such a network adapts to a new one from its calibration rows alone, as conditional neural processes and prior-fitted networks do for regression and tabular prediction [8, 26, 15].

The problem imposes two structural requirements. The predictor must be *mask-compatible*: defined for every subset of observed sensors, including subsets absent from training. It must also be *width-agnostic*: usable for any number of sensors. A product of per-sensor experts over the target meets both construction requirements, because a missing sensor simply contributes no factor [13, 36]. Learned message passing in the style of expectation propagation (EP) refines such experts using context from the other sensors [24, 12].

Contributions.

1. **When products of experts fail (§3–4).** Under shared noise, no product of mask-independent experts over the target alone is Bayes-consistent across missingness patterns; the optimal weight of a sensor can even change sign with the other sensors present. Under the generic-rank assumptions of Proposition 3, exact static linear-Gaussian sites require at least K nuisance dimensions for rank-K shared noise. These results do not exclude mask- or cavity-conditioned scalar sites.
2. **Lifted cavity networks (§4).** Each observed sensor contributes a rank-one Gaussian site over the target and a low-dimensional shared nuisance, and missing sensors are omitted exactly. A support-fitted factor-analysis anchor initializes either a small cavity-refinement network or a transformer with a learned site output layer.
3. **A pre-registered evaluation (§6)** against exact posteriors, a Hamiltonian Monte Carlo reference conditioned on support-inferred response signs, tuned closed forms and a TabPFN-v2-style transformer trained on the same task distribution, on synthetic panels and two real sensor networks. Three protocols were committed before their test data existed, and every endpoint is reported.

What we find. Structure buys sample efficiency and extrapolation, but on real data it costs flexibility.

- On synthetic tasks, the 8k-parameter lifted cavity network beats a 203k-parameter TabPFN-v2-style transformer trained on the same tasks for every number of missing sensors, closing 58% of the HMC reference gap against 34%; its margin grows on 16 sensors after training on five. It loses its edge under strong response nonlinearity and on linear sensors (§6.3).
- On two real sensor networks, the fine-tuned lifted network improves on the closed forms, but a fine-tuned transformer is better, and one pre-registered endpoint fails (§6.5).
- As the transformer's output layer (§6.6), lifted sites beat the plain transformer on fresh synthetic panels and the lifted network on a new real target, recovering most of the real-data gap (75% for the pre-registered seed, 62% averaged over two); non-inferiority to the fine-tuned transformer is not established.

2 Problem setup

A task (deployment) $\tau \sim p(\tau)$ defines a joint distribution $p_\tau(x, y)$ over P sensor readings $x \in \mathbb{R}^P$ and a scalar target y. The support $\mathcal{D} = \{(x_i, m_i, y_i)\}_{i=1}^n$ holds n labeled rows whose entry x_{ij} is observed where $m_{ij} = 1$. A query is a reading x_S of an observed subset $S \subseteq \{1, \dots, P\}$. The target of prediction is the posterior predictive

$$p(y \mid x_S, \mathcal{D}) = \int p_\tau(y \mid x_S)\, p(\tau \mid \mathcal{D}, x_S)\, d\tau, \tag{1}$$

and methods are scored by the negative log-likelihood (NLL) of the held-out target. Two privileged references bound what is possible on synthetic tasks: the *oracle* $p_\tau(y \mid x_S)$, which knows the task, and the *Bayes-optimal predictive* (1), which knows only the task prior. Our numerical HMC reference approximates the latter after conditioning on response signs inferred from the support; it is not a certified Bayes-optimal ceiling. Reported gap-closure percentages use the NLL difference between the closed-form FA-Gaussian of §4 and this sign-conditioned HMC reference (the *reference gap*).

An in-context method learns a map $(\mathcal{D}, x_S) \mapsto q(y)$ by minimizing NLL over tasks drawn from a training distribution [8, 26, 9]. In synthetic training, query masks omit *at most one* sensor; evaluation

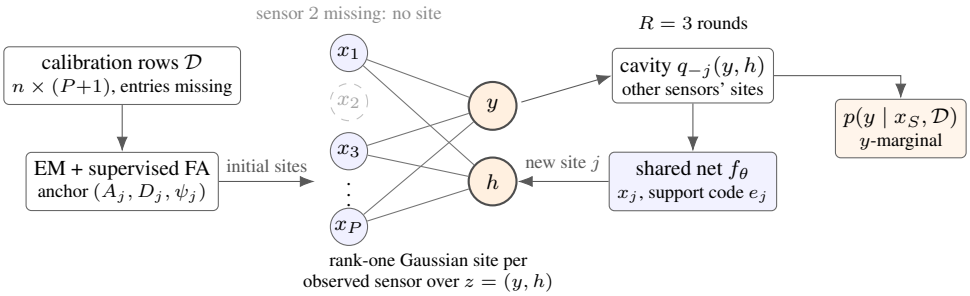


Figure 1: Lifted cavity network. A supervised factor analysis estimated from the incomplete calibration rows gives every sensor a rank-one Gaussian site over the target y and a shared nuisance h. Missing sensors contribute no site, so any subset and any number of sensors are handled by omission. For R rounds, a shared network replaces each observed sensor's site using its cavity (the posterior from the other sensors) and an encoding of that sensor's calibration scatter. The prediction is the y-marginal of the product.

also deletes two or three. These synthetic deletion counts are unseen in training. Real-data fine-tuning retains natural missingness and removes at most one additional sensor, so it can contain several missing sensors; unseen-count claims do not apply to those real episodes.

3 Scalar experts cannot fuse sensors with shared noise

Consider $x_j = a_j y + d_j u + \sigma_j \epsilon_j$ with mutually independent $y, u, \epsilon_j \sim \mathcal{N}(0, 1)$, $\sigma_j > 0$ and $a_j \neq 0$. Here u is a nuisance shared by all sensors. A product-of-experts (PoE) predictor gives each sensor a factor $t_j(y; x_j)$ and predicts $q(y \mid x_S) \propto p(y) \prod_{j \in S} t_j(y; x_j)$. Omitting missing sensors is what makes the construction mask-compatible.

Proposition 1 (Scalar experts)**.** *Let each t_j be Gaussian in y with positive precision, depend on x_j, and not depend on S. Suppose q equals the Bayes posterior for every reading in each singleton query $S = \{j\}$. For a pair $\{j, k\}$, q equals the Bayes posterior for every reading if and only if $d_j d_k = 0$. Moreover, q's pair posterior mean gives x_j a coefficient with the sign of a_j, whereas the Bayes coefficient can have the opposite sign.*

The proof (Appendix A) shows that matching single-sensor queries forces $t_j(y; x_j) \propto p(x_j \mid y)$, so q is the naive-Bayes posterior. For example, take $a = (1, 1)$, $d = (1, 0.5)$ and $\sigma = (0.05, 0.05)$. The Bayes coefficient of x_1 is $+0.50$ when x_1 is alone and -0.96 in the pair: the second sensor lets the predictor estimate u and subtract it from the first.

EP [24] refines each factor using its *cavity*, the approximate posterior given the other sensors, and learned EP amortizes the refinement [12, 6, 16]. With a Gaussian over y alone, however, the cavity summarizes the other sensors by a scalar mean and variance, discarding what they reveal about u. Proposition 1 does not exclude every scalar-cavity network, but the failure is visible in practice (§6.4).

4 Lifted cavity networks

Lifted sites. Let $z = (y, h) \in \mathbb{R}^{1+K}$, with independent $h \sim \mathcal{N}(0, I)$ and $y \sim \mathcal{N}(\mu_y, v_y)$, $v_y > 0$. Write the prior in natural parameters (Λ_0, η_0). Each observed sensor contributes a rank-one Gaussian site, which corresponds to a linear-Gaussian measurement $\tilde{x}_j = w_j^\top z + e_j$ with $e_j \sim \mathcal{N}(0, \psi_j)$:

$$\Lambda_j = \frac{w_j w_j^\top}{\psi_j}, \qquad \eta_j = \frac{w_j \, o_j}{\psi_j}. \tag{2}$$

The prediction is the y-marginal of

$$q(z \mid x_S) = \mathcal{N}\big(z; \Lambda^{-1}\eta, \Lambda^{-1}\big), \qquad \Lambda = \Lambda_0 + \textstyle\sum_{j \in S} \Lambda_j, \quad \eta = \eta_0 + \textstyle\sum_{j \in S} \eta_j. \tag{3}$$

Proposition 2 (Exactness). *Suppose $x_j = A_j y + D_j^\top h + c_j + e_j$, with $\psi_j > 0$, mutually independent $e_j \sim \mathcal{N}(0, \psi_j)$ independent of z, and the prior above. Set $w_j = (A_j, D_j)$ and $o_j = x_j - c_j$. Then* (3) *equals $p(z \mid x_S)$ for every fixed $S \subseteq \{1, \dots, P\}$. Its y-marginal is Bayes predictive for noninformative observation masks; informative masks require a selection model.*

Shared noise of rank K is absorbed by K nuisance dimensions, so the obstruction of Proposition 1 disappears: the lifted product differences out h automatically. Under the following generic-rank condition, fewer nuisance dimensions cannot suffice for static linear-Gaussian sites:

Proposition 3 (Lifting dimension). *Let $x = ay + Du + e$ with mutually independent $y \sim \mathcal{N}(0, 1)$, $u \sim \mathcal{N}(0, I_K)$ and Gaussian errors e_j of positive variance, $D \in \mathbb{R}^{P \times K}$, and every $a_j \neq 0$. Suppose that after deleting any row, $L = [a, D]$ still contains two disjoint nonsingular $(1 + K) \times (1 + K)$ submatrices, which holds for almost every L once $P \geq 2K + 3$. If a product of mask-independent sites of the form* (2) *with $o_j = x_j - c_j$, fixed w_j, c_j and fixed $\psi_j > 0$ (all independent of readings and subsets), over $z = (y, h)$ with $h \in \mathbb{R}^{K'}$ and a proper nonsingular Gaussian prior, reproduces $p(y \mid x_S)$ for every reading and every S with $|S| \leq 2$, then $K' \geq K$.*

The proof (Appendix A) shows that the conditionals on single sensors and pairs identify the covariance of the sensors, which a product of such sites can only represent as a matrix of rank $1 + K'$ plus a diagonal; the identifiability argument of factor analysis [1] excludes this when $K' < K$. Our main synthetic panels ($K = 1$, $P = 5$) meet the condition, so for products of static sites one nuisance dimension is necessary and sufficient.

Closed-form initialization from the support. Before any learning, we estimate the factor model from $\mathcal{D}$ alone. EM for a Gaussian with missing entries [4, 21] gives the joint mean and covariance of (x, y) from the incomplete calibration rows. Then $A = \mathrm{Cov}(x, y)/\mathrm{Var}(y)$, and principal-factor iterations on the residual covariance $\mathrm{Cov}(x) - AA^\top \mathrm{Var}(y)$ give (D, ψ) [31, 33]. With sensors standardized by their support statistics, $o_j = \tilde{x}_j + A_j \mu_y$. We call the resulting predictor *FA-Gaussian*; it is exact for every mask under the fitted model.

Learned cavity refinement. Real sensors are nonlinear, real noise is not exactly factor-structured, and $n = 48$ calibration rows make plug-in estimates noisy. We therefore refine the sites for R rounds. In each round, every observed sensor j removes its site from (3) to form the cavity $q_{-j}(z) = \mathcal{N}(m_{-j}, V_{-j})$, the posterior over target and nuisance given the other sensors. The cavity predicts the reading as $\hat{x}_j = A_j(m_{-j,y} - \mu_y) + D_j^\top m_{-j,h}$, with variance $v_j = w_j^\top V_{-j} w_j + \psi_j$. A shared two-layer network f_θ reads the reading $\tilde{x}_j$, an encoding e_j of sensor j's calibration scatter, the anchor $(A_j, D_j, \log \psi_j)$, the cavity $(m_{-j}, \log V_{-j,yy}, V_{-j,yh})$, the raw and standardized residuals of $\tilde{x}_j$ against $\hat{x}_j$, $\log v_j$ and the fraction of observed sensors. It outputs a correction $(\delta_j^{\mathrm{off}}, \delta_j^{\mathrm{slope}}, \delta_j^D, s_j)$, and the site is replaced by the local linearization of the sensor's response at the cavity mean $\bar{y} = m_{-j,y}$:

$$\begin{aligned} w_j &= (A_j + \delta_j^{\mathrm{slope}},\, D_j + \delta_j^D), \quad & o_j &= \tilde{x}_j - g_j(\bar{y}) + w_{j,y}\bar{y}, \\ g_j(\bar{y}) &= A_j(\bar{y} - \mu_y) + \delta_j^{\mathrm{off}}, & \psi_j &\leftarrow \psi_j\, e^{4\tanh(s_j/4)}. \end{aligned} \tag{4}$$

The encoding $e_j \in \mathbb{R}^{16}$ is a DeepSets network [37] over sensor j's calibration scatter, so f_θ can recognize a sensor's nonlinearity from the support. The last layer of f_θ is initialized at zero.

If f_θ were replaced by the statistical linear regression of a *known* response function under the cavity, these rounds would be EP with posterior linearization [3, 7]. The network amortizes the unknown response from the calibration rows, in the spirit of algorithm unrolling [11, 25].

Proposition 4 (Structural guarantees). *For any parameters θ:*

(i) the prediction does not depend on the readings of missing sensors;
(ii) conditional on fully equivariant anchor parameters and support statistics with consistent nuisance orientation under calibration-column permutations, permuting sensors and their calibration columns leaves the prediction unchanged; the same θ applies for every P;
(iii) when the last layer of f_θ is zero, the network equals the FA-Gaussian closed form of its anchor, so the infimum of training risk over θ is at most that closed form's.

The zero-head property is representational: it guarantees neither optimization convergence nor that a trained checkpoint improves on the anchor on new tasks. The original learned model does not en-

force invariance to equivalent factor signs. Anchor refitting can therefore violate end-to-end sensor-permutation invariance through a changed nuisance orientation. A separately versioned one-factor sign-averaging repair passed source-only structural checks; the reported efficacy results remain those of the original model and do not establish the repaired model's efficacy.

Training. We minimize the predictive NLL over episodes drawn from a pool of 60,000 synthetic tasks (128 tasks $\times$ 8 queries per step, 20,000 AdamW steps with cosine decay, query masks with at most one missing sensor). We use $K = 1$ and $R = 3$. The model has about 8,000 parameters and trains in 15–20 minutes on one CPU core, 14 times less than the transformer baseline (Table 4). The same recipe trains every learned baseline except the transformer, which gets fresh tasks at every step.

Lifted sites as a transformer output layer. The *lifted cavity transformer* (LCT) uses the same cell-token backbone as the plain transformer, with a zero-initialized linear map from each observed query-cell token to the site corrections in (4). It linearizes once at the FA-Gaussian posterior mean, then returns the target marginal of the product of observed sites. Unlike the small network, it does not perform three cavity-refinement rounds. The backbone can condition site corrections on the full support and query subset; lifting constrains the probabilistic readout, not those interactions. Missing readings are excluded and a zero head recovers the anchor, so parts (i) and (iii) of Proposition 4 carry over; part (ii) is not asserted for the transformer's random column embeddings. Given the tokens, site aggregation and the $(K+1)$-dimensional solve cost $O(P(K+1)^2 + (K+1)^3)$ per query; this excludes the backbone and support-anchor fitting. The contrast with a plain transformer tests this readout as a package: it changes both the head parameterization and initialization, without separately attributing their effects.

5 Related work

Products of experts handle missing modalities by omission [13, 36, 22]; our negative result concerns scalar-target experts; MVAE already uses multivariate latent products. **Learned message passing**, through learned EP messages [12, 6, 16] or unrolled inference [11, 25], learns reusable operators for specified factors; here sensor response is inferred from deployment calibration support, with a target/nuisance cavity. **Supervised learning with missing values** [30, 21, 17] includes task-specific predictors; NeuMiss [20] unrolls the Bayes predictor of a Gaussian model but is trained per dataset and keyed to feature identities. **In-context prediction** with neural processes [8, 19] and prior-fitted networks such as TabPFN and TabICL [26, 14, 15, 28] uses generic set or transformer architectures (the cited TabICL is classification-only). Our tested transformer is trained on our task distribution; pretrained comparisons are pending. **Fusion under unknown correlations** [18, 35, 5, 29] includes learned sequential state-space filtering and decentralized network fusion. DIFNet explicitly discusses changing sensor nodes and network topologies. Our distinction is episodic prediction with unknown sensor calibration inferred from a labeled support set, rather than sequential filtering in a specified state-space model.

6 Experiments

6.1 Setup

Synthetic tasks. Each task draws, for every sensor j, a response $x_j = a_j y + b_j + 0.4\sin(f_j y) + d_j u + \sigma_j \epsilon_j$ with $a_j = \pm\mathcal{U}[0.6, 1.8]$ (random sign), $b_j \sim \mathcal{N}(0, 0.4^2)$, $f_j \sim \mathcal{U}[0.5, 2.5]$, shared-noise loading $d_j \sim \mathcal{N}(0, 0.7^2)$, $\sigma_j \sim \mathcal{U}[0.15, 0.8]$ and $y, u, \epsilon_j \sim \mathcal{N}(0, 1)$. The support has $n = 48$ rows with 20% of sensor entries missing at random, and 48 query rows are scored under every mask with k missing sensors (or 20 seeded masks when there are more). The main panel (F1) has 256 fresh tasks with five sensors. Six further panels of 128 tasks change one factor at a time: 8 or 16 sensors, nonlinearity 0.8 or 0, and 16 or 96 support rows.

Methods. *Support-only closed forms:* FA-Gaussian, EM-Gaussian, NL-FA (cubic responses with a tuned curvature penalty and exact grid inference, a strong plug-in nonlinear model), evidence-tuned Bayesian linear regression (BLR), ridge, and, post hoc, a Gaussian process. *Learned in-*

Table 1: Main synthetic panel (F1: 256 fresh tasks, five sensors). Mean test NLL by number of missing query sensors; learned models saw at most one missing sensor in training, so $k = 2, 3$ are unseen patterns. Learned rows average three training seeds (seed s.d. ≤ 0.002) except the transformers (seed 1 as pre-registered; Table 11 adds a second seed). Last column: share of the reference gap (FA-Gaussian to sign-conditioned HMC) closed at $k = 2$. Best non-privileged entry per column in bold. †Post hoc, scored at $k = 2$ only. ‡Designed after protocol v2 (Section 6.6); descriptive on this panel and not bolded.

Method	k=0	k=1	k=2	k=3	Gap closed
Privileged references					
Oracle (true parameters)	−0.419	−0.250	−0.026	0.287	–
Sign-conditioned HMC reference	−0.334	−0.174	0.039	0.337	–
Support-only closed forms					
FA-Gaussian	−0.152	−0.011	0.177	0.439	0%
EM-Gaussian	−0.048	0.037	0.195	0.441	−13%
NL-FA (cubic, tuned)	−0.173	−0.036	0.151	0.417	19%
Bayesian linear regression	0.111	0.091	0.214	0.450	−26%
Ridge (complete case)	0.026	0.078	0.216	0.451	−28%
Ridge (mean-imputed)	0.198	0.280	0.399	0.582	−161%
GP, complete case†	–	–	0.416	–	−173%
Learned in context (trained on $k \leq 1$)					
Lifted cavity (ours), 8k	**−0.265**	**−0.112**	**0.097**	**0.395**	58%
Transformer (TabPFN-v2 style), 203k	−0.246	−0.084	0.131	0.424	34%
Residual MLP on FA anchor, 15k	−0.236	−0.087	0.114	0.397	46%
Scalar-cavity network, 3k	0.055	0.178	0.336	0.555	−115%
Transformer, trained on all masks	−0.237	−0.076	0.132	0.411	33%
Lifted cavity transformer (v3)‡, 203k	−0.265	−0.112	0.095	0.387	59%

context models, all trained on the same task family and masks: our lifted cavity network (8k parameters, three seeds); a residual MLP that receives the same FA anchor and support encodings (15k parameters, five sensors only); the scalar-cavity network of §3; and a TabPFN-v2-style cell-token transformer [15] trained on fresh tasks at every step (203k parameters, 30,000 steps). A second transformer was trained on *all* masks, to separate architecture from training coverage. *Privileged references:* the oracle, and a sign-conditioned HMC approximation to the posterior predictive with the shared nuisance marginalized analytically (Appendix C).

Protocol. NLL is the primary metric; coverage, CRPS [10] and RMSE are secondary. Intervals are paired 95% intervals over tasks, with per-task NLL averaged over training seeds. Three evaluation protocols were committed to version control before their panels existed, and every endpoint is reported (Table 5, Appendix D).

6.2 Synthetic tasks with exact references

Table 1 and Figure 2a report the main panel. The lifted cavity network is the best non-privileged method for every number of missing sensors, including the two- and three-sensor deletions it never saw in training. With two missing sensors it closes 58% of the reference gap between FA-Gaussian and the sign-conditioned HMC approximation. Two comparators close less: the residual MLP, which receives the same anchor and support encodings with twice the parameters (46%), and the tuned nonlinear plug-in model NL-FA (19%). The TabPFN-v2-style transformer has 25 times the parameters and sees fresh tasks at every step, yet it is worse than the lifted cavity network for every number of missing sensors, by 0.020–0.033 nats (all intervals exclude zero), and closes 34% of the gap. Training it on every mask changes nothing (33%), so its deficit is learning efficiency, not unseen patterns. The residual MLP is the closest competitor: the lifted network beats it by 0.030 nats with all sensors observed and 0.016 with two missing, and the two tie with three missing. EM-Gaussian, BLR and both ridges fall below FA-Gaussian; mean imputation is the worst of all.

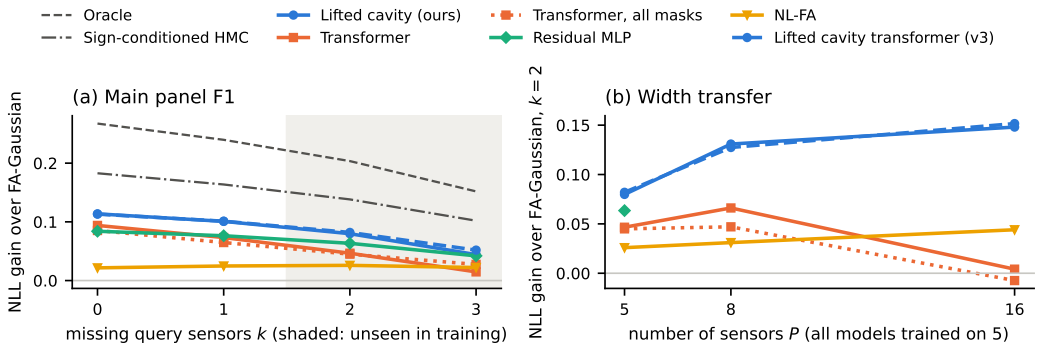


Figure 2: NLL gain over the closed-form FA-Gaussian anchor (higher is better). (a) Main panel F1 by number of missing query sensors; shaded patterns were never seen in training. Grey lines are the privileged oracle and the sign-conditioned HMC reference. (b) Width transfer with two missing sensors: every learned model was trained on five sensors, and the residual MLP exists only at $P = 5$. Dashed blue: the lifted cavity transformer of §6.6, descriptive on these panels.

6.3 Generalization beyond the training distribution

Width (Table 7, Figure 2b). The lifted cavity network runs unchanged on 8 and 16 sensors, where it improves on FA-Gaussian by $+0.131$ [$+0.097$, $+0.165$] and $+0.148$ [$+0.115$, $+0.181$] nats with two missing sensors. The residual MLP cannot run on these panels, and EM-Gaussian degrades as the covariance grows. The transformer also runs at any width, but it gains less over FA-Gaussian as sensors are added. On 16 sensors it is no better than the closed form, and the lifted cavity network beats it by $+0.144$ [$+0.127$, $+0.161$] nats. **Support size.** With 16 calibration rows the transformer degrades sharply, while the lifted cavity network keeps a $+0.146$ [$+0.073$, $+0.219$] nat gain over FA-Gaussian; with 96 rows it remains the best method. **Stronger nonlinearity** (0.8 instead of 0.4) is the one shift where the transformer competes. It ties the lifted cavity network with two missing sensors (difference $+0.000$ [-0.011, $+0.012$]) and is better by 0.041 nats when all sensors are observed: under strong response nonlinearity alone, Gaussian re-linearization offers no advantage over a generic transformer. **Linear sensors.** When the true sensors are linear, FA-Gaussian is nearly the right model and is best; the learned re-linearization then costs 0.022 nats with two missing sensors, and the transformer 0.076.

6.4 What the lifting buys

Ablations on F1 (Table 8, Appendix F) compare components of the small network. **Lifting** is the largest factor: scalar sites ($K = 0$) with otherwise identical machinery and training lose 0.134 [$+0.120$, $+0.148$] nats with two missing sensors and end up worse than the untrained FA-Gaussian anchor. This supports lifting empirically under the tested training regime; Proposition 1 does not cover this context-conditioned ablation. **Cavity conditioning** adds 0.026 [$+0.022$, $+0.029$] nats on top of lifted static sites. A second nuisance dimension ($K = 2$) changes nothing, as expected for one-factor noise. The scalar-cavity network of the earlier design, retrained with the same 60,000-task recipe, remains far behind FA-Gaussian (Appendix D).

6.5 Real sensor networks

Beijing multi-site air quality [38]. The task is to predict log $PM_{2.5}$ at each of 12 monitoring stations from the other 11, using hourly data with natural missingness. This tests contemporaneous missing-station interpolation, rather than low-cost-device calibration or future forecasting. Each episode is one station and one week, with 48 labeled support hours and 48 query hours. The test period (2016-01 to 2017-02, 718 episodes) was never examined before protocol v2 was committed. Learned models are fine-tuned on episodes from 2013–2015 with one fixed recipe; the protocol's analysis is cluster-robust over weeks.

Table 2: Real sensor networks: mean test NLL on later, held-out periods. Beijing: 12 target stations $\times$ weekly episodes in 2016–2017 with natural missingness, plus extra dropped sensors. Air Quality: weekly episodes after October 2004, sensor dropout simulated. Fine-tuning uses only earlier periods and one recipe for every model; lifted-cavity and residual-MLP rows average three seeds. Best entry per column in bold. †Post hoc. ‡Designed after protocol v2 (Section 6.6); post hoc here and not bolded.

Method	Beijing $PM_{2.5}$ (11 stations)			Air Quality CO		Air Quality NO_2	
	natural	+3	+6	k=0	k=2	k=0	k=2
EM-Gaussian	1.032	0.742	0.569	0.074	0.085	−0.237	−0.175
FA-Gaussian	0.677	0.615	0.562	0.052	0.116	−0.196	−0.155
NL-FA	0.938	0.744	0.610	0.144	0.091	−0.090	−0.142
Bayesian linear regression	0.399	0.406	0.442	1.037	0.216	−0.055	−0.178
Ridge (complete case)	0.425	0.432	0.458	0.824	0.208	−0.200	−0.176
GP, complete case†	1.884	1.312	0.934	3.364	0.503	25.217	−0.198
k-nearest neighbours†	0.508	0.514	0.540	0.571	0.353	−0.010	−0.063
Lifted cavity, zero-shot	0.826	0.661	0.561	0.044	0.129	−0.350	−0.233
Transformer, zero-shot	0.958	0.743	0.687	−0.017	0.081	−0.360	−0.231
Lifted cavity, fine-tuned (ours)	0.205	0.237	0.305	−0.040	0.008	−0.412	−0.337
no synthetic pretraining	0.247	0.279	0.344	−0.001	−0.006	−0.419	−0.316
no cavity input	0.259	0.286	0.349	−0.086	−0.028	−0.423	−0.332
scalar sites ($K = 0$)	0.215	0.252	0.347	−0.026	−0.003	−0.359	−0.302
Transformer, fine-tuned	**0.087**	**0.121**	**0.236**	**−0.163**	**−0.052**	**−0.485**	**−0.381**
Residual MLP, fine-tuned	–	–	–	0.025	0.110	−0.246	−0.147
Lifted cavity transformer, fine-tuned‡	0.122	0.160	0.259	−0.099	−0.027	−0.404	−0.359

With 11 sensors and 48 rows, the joint-covariance closed forms overfit, and evidence-tuned BLR is the strongest closed form. The fine-tuned lifted cavity network improves on it by 0.194 [+0.131, +0.257] nats under natural missingness, and by 0.137 [+0.104, +0.170] with six further sensors removed. The fine-tuned transformer is better still. The lifted cavity network's gain over it is −0.118 [−0.145, −0.091] nats under natural missingness and −0.069 [−0.080, −0.058] with six sensors removed, so endpoint E4b fails. Before fine-tuning the order is reversed: zero-shot, the lifted cavity network is ahead by 0.127 [+0.057, +0.197] nats with six sensors removed. Synthetic pretraining helps: the same network fine-tuned from initialization is 0.041 nats worse [+0.033, +0.050].

UCI Air Quality [2]. Five metal-oxide sensors on one device predict the reference analyzers' log CO and log NO_2, in weekly episodes after October 2004. These weeks were examined in protocol v1, so this is secondary evidence. With two simulated missing sensors, the fine-tuned lifted cavity network improves on FA-Gaussian by +0.107 [+0.015, +0.200] nats for CO and +0.181 [+0.076, +0.287] for NO_2. The fine-tuned transformer is again better: the lifted network's gain over it is −0.060 [−0.106, −0.014] (CO) and −0.044 [−0.074, −0.014] (NO_2). On CO, the cavity input stops helping after fine-tuning.

Why the transformer wins on real data. The transformer is better on 89% of Beijing test episodes, by 0.15–0.16 nats in each of the three easier quartiles of closed-form difficulty. The lifted network underfits rather than overfits: the transformer's final training NLL is 0.19 nats lower, about the test gap. Neither a second nuisance dimension (0.205) nor five times more fine-tuning steps (0.169, against 0.207; Table 10) closes the gap to the transformer (0.087), and per-episode nonparametrics do worse still (k-nearest neighbours 0.508, Gaussian process 1.884). We therefore hypothesized that the limitation lies in how sites are refined (each correction sees only its own sensor's reading, calibration scatter and cavity) rather than in the site family; protocol v3 tests this.

6.6 Lifted sites as a transformer output layer (protocol v3)

Protocol v2 left a split: structure wins on synthetic tasks, the transformer on real networks after fine-tuning. The *lifted cavity transformer* (LCT; §4) keeps the transformer's backbone, recipe and seed, with a lifted probabilistic readout. Because the hypothesis was formed after the v2 results, v3 uses new panels (G1, five sensors; G3, sixteen) and two Beijing pollutants never scored before (NO_2 and CO), and its four endpoints were committed before any of these existed (Table 5).

On the fresh synthetic panels, the lifted output layer helps the transformer (Table 3). With two missing sensors the LCT beats the transformer with the identical backbone, recipe and seed by 0.035

Table 3: Protocol v3 (new panels and targets). Mean test NLL. G1: 256 fresh five-sensor tasks by number of missing query sensors k; G3: 128 fresh 16-sensor tasks, $k = 2$. Beijing: log NO_2 and log CO at each station from the other 11 stations, 2016–2017, natural missingness and six further sensors removed; learned models fine-tuned with the recipe of protocol v2. Best entry per column in bold.

	G1, P=5				G3	Beijing NO_2		Beijing CO	
Method	k=0	k=1	k=2	k=3	P=16	natural	+6	natural	+6
FA-Gaussian	−0.189	−0.054	0.129	0.396	−0.806	0.684	0.506	0.384	0.334
Bayesian linear regression	0.013	0.022	0.153	0.401	0.063	0.275	0.365	0.170	0.248
Lifted cavity network, 8k	**−0.329**	**−0.172**	0.040	0.348	−0.934	0.282	0.334	0.149	0.229
Transformer, 203k	−0.315	−0.147	0.073	0.378	−0.800	**0.199**	0.296	**0.019**	**0.176**
Lifted cavity transformer, 203k	**−0.329**	**−0.172**	**0.039**	**0.340**	**−0.950**	0.220	**0.291**	0.054	0.187

[+0.027, +0.042] nats on G1 and 0.150 [+0.133, +0.167] on sixteen sensors (E5, E6 pass). It matches the small lifted network on five sensors (+0.002 [−0.002, +0.006]) and exceeds it on sixteen (+0.017 [+0.004, +0.029]), and it learns faster: its training NLL reaches the transformer's final value after 10,750 of 30,000 steps (Figure 3). On the v2 panels (descriptive) it closes 59% of the F1 gap (lifted network 58%) but inherits part of the transformer's degradation with 16 calibration rows. After fine-tuning on Beijing NO_2, it beats the fine-tuned lifted network by +0.062 [+0.024, +0.100] nats (E7a passes), recovering 75% of the gap between the lifted network (0.282) and the transformer (0.199). Against the transformer the difference is −0.021 [−0.093, +0.052]; the interval includes zero but extends below the margin of −0.02, so non-inferiority (E7b) is not established. With six more stations missing, the two tie (+0.005 [−0.027, +0.037]). The real-data gain is seed-sensitive: with second seeds of both transformers (post hoc) the LCT recovers 50%, and 62% averaged over seeds (Table 11). The pattern repeats on Beijing CO (descriptive) and $PM_{2.5}$ (post hoc), with 73% and 70% of the gap recovered; on the small Air Quality device (post hoc) it is about half with two missing sensors (CO 58%, NO_2 50%) and none for NO_2 with all sensors.

6.7 Pre-registered endpoints and calibration

Table 5 in Appendix D lists every declared endpoint; v4 decisions are withheld after its provenance gate failed. Five of six v2 endpoints pass; the failure, E4b, is the real-data comparison with the fine-tuned transformer. All five v1 endpoints passed (Appendix D). Three of the four v3 endpoints pass; the failure, E7b, is non-inferiority to the fine-tuned transformer on Beijing NO_2, where the interval is wide. Learned models are close to nominal 90% coverage (lifted cavity network 0.901 on F1, 0.914 on Beijing), while the closed forms under-cover (Table 9).

7 Limitations

Scope. Exact oracle and sign-conditioned HMC references exist only for our generator (one-factor noise, sinusoidal nonlinearity, Gaussian targets). Per-sensor dropout is simulated on Air Quality and mostly simulated on Beijing, whose natural missingness covers 1.6% of query readings; the gas-drift benchmark could not separate methods (Appendix D). **Model class.** Gaussian sites cannot represent multimodal posteriors; we used $K = 1$ throughout. **Baselines.** The reported transformers use 203k parameters and 30,000 steps; second seeds reproduce the synthetic comparisons, but the LCT's real-data gain varies with the seed. A three-seed 2.1M-parameter GPU study completed, but missing checkpoint bindings blocked independent verification (Appendix D). Its endpoints and pretrained TabPFN v2 efficacy remain unverified; TabICL is unevaluated.

8 Conclusion

Target/nuisance sites represent factor noise exactly under our static linear-Gaussian assumptions. The small trained model outperforms the tested larger transformer on synthetic tasks and extrapolates across sensor counts, but loses on real networks. As the transformer's output layer, lifted sites keep the extrapolation and recover much of that gap; closing the rest, richer site families and vector targets are natural next steps.

References

[1] T. W. Anderson and Herman Rubin. Statistical inference in factor analysis. In *Proceedings of the Third Berkeley Symposium on Mathematical Statistics and Probability*, volume 5, pages 111–150, 1956.

[2] Saverio De Vito, Ettore Massera, Marco Piga, Luca Martinotto, and Girolamo Di Francia. On field calibration of an electronic nose for benzene estimation in an urban pollution monitoring scenario. *Sensors and Actuators B: Chemical*, 129(2):750–757, 2008.

[3] Marc Peter Deisenroth and Shakir Mohamed. Expectation propagation in Gaussian process dynamical systems. In *Advances in Neural Information Processing Systems*, volume 25, 2012.

[4] Arthur P. Dempster, Nan M. Laird, and Donald B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B*, 39(1):1–22, 1977. doi: 10.1111/j.2517-6161.1977.tb01600.x.

[5] Ruifeng Dong, Ming Wang, Ning Liu, Tong Guo, Jiayi Kang, Xiaojing Shen, and Yao Mao. DIFNet: Decentralized information filtering fusion neural network with unknown correlation in sensor measurement noises. *arXiv preprint arXiv:2508.18854*, 2025.

[6] S. M. Ali Eslami, Daniel Tarlow, Pushmeet Kohli, and John Winn. Just-in-time learning for fast and flexible inference. In *Advances in Neural Information Processing Systems*, volume 27, 2014.

[7] Ángel F. García-Fernández, Lennart Svensson, Mark R. Morelande, and Simo Särkkä. Posterior linearization filter: Principles and implementation using sigma points. *IEEE Transactions on Signal Processing*, 63(20):5561–5573, 2015.

[8] Marta Garnelo, Dan Rosenbaum, Christopher Maddison, Tiago Ramalho, David Saxton, Murray Shanahan, Yee Whye Teh, Danilo Rezende, and S. M. Ali Eslami. Conditional neural processes. In *Proceedings of the 35th International Conference on Machine Learning*, pages 1704–1713, 2018.

[9] Samuel J. Gershman and Noah D. Goodman. Amortized inference in probabilistic reasoning. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, volume 36, 2014.

[10] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

[11] Karol Gregor and Yann LeCun. Learning fast approximations of sparse coding. In *Proceedings of the 27th International Conference on Machine Learning*, pages 399–406, 2010.

[12] Nicolas Heess, Daniel Tarlow, and John Winn. Learning to pass expectation propagation messages. In *Advances in Neural Information Processing Systems*, volume 26, 2013.

[13] Geoffrey E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771–1800, 2002.

[14] Noah Hollmann, Samuel Müller, Katharina Eggensperger, and Frank Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. In *International Conference on Learning Representations*, 2023.

[15] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637:319–326, 2025.

[16] Wittawat Jitkrittum, Arthur Gretton, Nicolas Heess, S. M. Ali Eslami, Balaji Lakshminarayanan, Dino Sejdinovic, and Zoltán Szabó. Kernel-based just-in-time learning for passing expectation propagation messages. In *Proceedings of the 31st Conference on Uncertainty in Artificial Intelligence (UAI)*, 2015.

[17] Julie Josse, Nicolas Prost, Erwan Scornet, and Gaël Varoquaux. On the consistency of supervised learning with missing values. *arXiv preprint arXiv:1902.06931v1*, 2019. URL `https://arxiv.org/abs/1902.06931v1`.

[18] Simon J. Julier and Jeffrey K. Uhlmann. A non-divergent estimation algorithm in the presence of unknown correlations. In *Proceedings of the American Control Conference*, volume 4, pages 2369–2373, 1997.

[19] Hyunjik Kim, Andriy Mnih, Jonathan Schwarz, Marta Garnelo, Ali Eslami, Dan Rosenbaum, Oriol Vinyals, and Yee Whye Teh. Attentive neural processes. In *International Conference on Learning Representations*, 2019.

[20] Marine Le Morvan, Julie Josse, Thomas Moreau, Erwan Scornet, and Gaël Varoquaux. NeuMiss networks: differentiable programming for supervised learning with missing values. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[21] Roderick J. A. Little and Donald B. Rubin. *Statistical Analysis with Missing Data*. Wiley, 3 edition, 2019.

[22] Mengmeng Ma, Jian Ren, Long Zhao, Sergey Tulyakov, Cathy Wu, and Xi Peng. SMIL: Multimodal learning with severely missing modality. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 2302–2310, 2021.

[23] David J. C. MacKay. Bayesian interpolation. *Neural Computation*, 4(3):415–447, 1992.

[24] Thomas P. Minka. Expectation propagation for approximate Bayesian inference. In *Proceedings of the 17th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 362–369, 2001.

[25] Vishal Monga, Yuelong Li, and Yonina C. Eldar. Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing. *IEEE Signal Processing Magazine*, 38(2):18–44, 2021.

[26] Samuel Müller, Noah Hollmann, Sebastian Pineda Arango, Josif Grabocka, and Frank Hutter. Transformers can do Bayesian inference. In *International Conference on Learning Representations*, 2022.

[27] Radford M. Neal. MCMC using Hamiltonian dynamics. In Steve Brooks, Andrew Gelman, Galin Jones, and Xiao-Li Meng, editors, *Handbook of Markov Chain Monte Carlo*. Chapman and Hall/CRC, 2011.

[28] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. TabICL: A tabular foundation model for in-context learning on large data. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, pages 50817–50847, 2025.

[29] Guy Revach, Nir Shlezinger, Xiaoyong Ni, Adrià López Escoriza, Ruud J. G. van Sloun, and Yonina C. Eldar. KalmanNet: Neural network aided Kalman filtering for partially known dynamics. *IEEE Transactions on Signal Processing*, 70:1532–1547, 2022.

[30] Donald B. Rubin. Inference and missing data. *Biometrika*, 63(3):581–592, 1976.

[31] Donald B. Rubin and Dorothy T. Thayer. EM algorithms for ML factor analysis. *Psychometrika*, 47(1):69–76, 1982.

[32] Lucas Theis, Aäron van den Oord, and Matthias Bethge. A note on the evaluation of generative models. In *International Conference on Learning Representations*, 2016.

[33] Michael E. Tipping and Christopher M. Bishop. Probabilistic principal component analysis. *Journal of the Royal Statistical Society: Series B*, 61(3):611–622, 1999.

[34] Alexander Vergara, Shankar Vembu, Tuba Ayhan, Margaret A. Ryan, Margie L. Homer, and Ramón Huerta. Chemical gas sensor drift compensation using classifier ensembles. *Sensors and Actuators B: Chemical*, 166–167:320–329, 2012.

[35] M. Wang, H. Liu, H. Tang, M. Zhang, and X. Shen. IFNet: Data-driven multisensor estimate fusion with unknown correlation in sensor measurement noises. *Information Fusion*, 115: 102750, 2025. doi: 10.1016/j.inffus.2024.102750.

[36] Mike Wu and Noah Goodman. Multimodal generative models for scalable weakly-supervised learning. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[37] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. Deep sets. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[38] Shuyi Zhang, Bin Guo, Anlan Dong, Jing He, Ziping Xu, and Song Xi Chen. Cautionary tales on air-quality improvement in Beijing. *Proceedings of the Royal Society A*, 473(2205): 20170457, 2017.

[39] Naomi Zimmerman, Albert A. Presto, Sriniwasa P. N. Kumar, Jason Gu, Aliaksei Hauryliuk, Ellis S. Robinson, Allen L. Robinson, and R. Subramanian. A machine learning calibration model using random forests to improve sensor performance for lower-cost air quality monitoring. *Atmospheric Measurement Techniques*, 11:291–313, 2018.

A Proofs

Proposition 1. Under the model, $p(x_j \mid y) = \mathcal{N}(x_j; a_j y, c_j)$ with $c_j = d_j^2 + \sigma_j^2$.

If $q(y \mid x_j) = p(y \mid x_j)$ for all x_j, then $p(y)\, t_j(y; x_j) \propto p(y)\, p(x_j \mid y)$ as functions of y. Hence $t_j(y; x_j) = \kappa_j(x_j)\, p(x_j \mid y)$, which is Gaussian in y with precision $a_j^2/c_j > 0$. For a pair, q is therefore the naive-Bayes posterior $p(y)\, p(x_j \mid y)\, p(x_k \mid y)$. Its posterior mean is linear in (x_j, x_k) with coefficients

$$(a_j/c_j,\ a_k/c_k)\,/\,\pi_q, \qquad \pi_q = 1 + a_j^2/c_j + a_k^2/c_k,$$

so the coefficient of x_j has the sign of a_j.

The Bayes posterior uses $C = \mathrm{Cov}(x_j, x_k \mid y)$, with off-diagonal entry $d_j d_k$. Its mean coefficients are $(C^{-1}a)^\top/\pi$, with $\pi = 1 + a^\top C^{-1} a$. Both posteriors are Gaussian, so they coincide for all (x_j, x_k) if and only if $C^{-1}a = \mathrm{diag}(C)^{-1}a$. Equivalently $a = C\,\mathrm{diag}(C)^{-1}a$, whose off-diagonal components are $d_j d_k\, a_k/c_k = 0$ and $d_j d_k\, a_j/c_j = 0$. With $a_j a_k \neq 0$ this holds if and only if $d_j d_k = 0$.

For the sign flip, take $a = (1, 1)$, $d = (1, 0.5)$ and $\sigma = (0.05, 0.05)$. Then the Bayes coefficient of x_1 is $+0.499$ alone and -0.959 in the pair. □

Proposition 2. Given $z = (y, h)$, the readings are independent: $x_j \mid z \sim \mathcal{N}(w_j^\top z + c_j, \psi_j)$. Hence

$$p(z \mid x_S) \propto p(z) \prod_{j \in S} \mathcal{N}(x_j; w_j^\top z + c_j, \psi_j).$$

In natural parameters each factor contributes $w_j w_j^\top/\psi_j$ to the precision and $w_j(x_j - c_j)/\psi_j$ to the shift, which is (2) with $o_j = x_j - c_j$. The prior is Gaussian, so the product is the Gaussian (3). This holds for every S because the factors of unobserved sensors are absent from both sides. Marginalizing h gives $p(y \mid x_S)$. □

Proposition 3. *Step 1: the product is a latent-factor model.* By the computation in the proof of Proposition 2 read backwards, a product of sites (2) with $o_j = x_j - c_j$ is the exact posterior over z of the model M_q: $z \sim \mathcal{N}(\mu_z, \Sigma_z)$ and, independently given z, $x_j \sim \mathcal{N}(w_j^\top z + c_j, \psi_j)$. Its y-marginal is M_q's conditional $p_q(y \mid x_S)$, and M_q's sensor covariance is $\Sigma_q = W \Sigma_z W^\top + \mathrm{diag}(\psi)$ with $W \in \mathbb{R}^{P \times (1+K')}$, so $\Sigma_q - \mathrm{diag}(\psi)$ has rank at most $1 + K'$.

Step 2: conditionals on $|S| \leq 2$ identify the covariance. Write $V = \mathrm{Var}(y)$, $c = \mathrm{Cov}(x, y)$ and $\Sigma = \mathrm{Cov}(x)$. The case $S = \emptyset$ gives V. For $S = \{j\}$ the conditional has slope $\beta_j = c_j/\Sigma_{jj}$ and variance $r_j = V - c_j\beta_j$, and $\beta_j \neq 0$ because $c_j = a_j \neq 0$. Hence $c_j = (V - r_j)/\beta_j$ and $\Sigma_{jj} = c_j/\beta_j$. For $S = \{j, k\}$ the slope vector $b \neq 0$ solves $\Sigma_{\{jk\}} b = (c_j, c_k)$, and whichever entry of b is non-zero determines Σ_{jk} from one row. Since M_q has the same conditionals, the same formulas apply to it, so $\Sigma_q = \Sigma = LL^\top + \mathrm{diag}(\sigma^2)$.

Step 3: fewer factors cannot reproduce Σ. Suppose toward contradiction $K' < K$. Let $r = 1 + K$, so $LL^\top$ has rank r. Suppose $\Sigma = G + \Psi'$ with Ψ' diagonal and $\mathrm{rank}(G) \leq r$. Fix a row i and two disjoint row sets I_1, I_2 not containing i with L_{I_1}, L_{I_2} nonsingular. In the $(r+1) \times (r+1)$ submatrix of Σ with rows $\{i\} \cup I_1$ and columns $\{i\} \cup I_2$, every entry except (i, i) lies off the diagonal, so it is shared by G and $LL^\top$. Both G and $LL^\top$ have rank at most r, so the corresponding minors vanish. Expanding along entry (i, i), its cofactor is $\det(L_{I_1} L_{I_2}^\top) \neq 0$, so $G_{ii} = (LL^\top)_{ii}$. Hence $G = LL^\top$, which has rank r [1]. Applying this to $G = W\Sigma_z W^\top$ from Step 1 gives $1 + K' \geq r = 1 + K$.

For almost every L all $r \times r$ submatrices are nonsingular, and $P \geq 2r + 1 = 2K + 3$ leaves two disjoint sets of r rows after deleting any row. □

Proposition 4.

(i) Sites of missing sensors are multiplied by the mask before entering (3), and the network's per-sensor inputs never include another sensor's raw reading, only cavity statistics built from observed sites.

(ii) Every operation is applied per sensor with shared weights. Conditional on a fully equivariant, consistently oriented anchor representation, its fitted sensor parameters permute with

the sensor coordinates, and sums over sensors are permutation-invariant. An unconstrained eigensolver can change factor signs or rotations when the anchor is refitted; the original learned network is not invariant to these changes. No input encodes a sensor's index or the total number of sensors, except the fraction of observed sensors, which is permutation invariant.

(iii) With a zero last layer, $\delta^{\text{off}} = \delta^{\text{slope}} = \delta^{D} = s = 0$. Then $w_j = (A_j, D_j)$, ψ_j is unchanged and $o_j = \tilde{x}_j - A_j(\bar{y} - \mu_y) + A_j\bar{y} = \tilde{x}_j + A_j\mu_y$. These are the initialization sites, so every round reproduces them and the output is the anchor's FA-Gaussian closed form. The zero point lies in the parameter set, so the infimum of the training risk is at most that closed form's. □

B Architecture and training details

Lifted cavity network ($K = 1$, $R = 3$, 8,004 parameters).

- **Support encoder.** $\phi : \mathbb{R}^3 \to \mathbb{R}^{32} \to \mathbb{R}^{32}$ with tanh, applied to each observed calibration entry $(\tilde{y}_i, \tilde{x}_{ij}, \tilde{r}_{ij})$. The inputs are the standardized target, the centred reading, and the anchor residual scaled by $\sqrt{\psi_j}$. The encoder averages over observed rows and then applies $\rho : \mathbb{R}^{33} \to \mathbb{R}^{16}$, where the extra input is the observation rate.
- **Site network.** $f_\theta : \mathbb{R}^{28} \to \mathbb{R}^{64} \to \mathbb{R}^{64} \to \mathbb{R}^4$ with tanh and a zero-initialized last layer.
- **Anchor.** EM uses 8 iterations with a 10^{-4} ridge. Principal factors use 40 iterations, with uniquenesses floored at 10^{-3}. Everything is computed in double precision from the support only.

Table 4: Compute. Training on one CPU core (core-hours); inference (ms/task) is single-threaded time per task for 480 queries (ten masks × 48 rows) on F1. Anchors are precomputed in batch and excluded; the closed-form rows include their unbatched NumPy fits.

Model	Parameters	Training steps	Core-hours	ms/task
Lifted cavity network	8,004	20,000	0.22	7.2
Residual MLP on FA anchor	15,490	20,000	0.20	1.2
Transformer (TabPFN-v2 style)	202,562	30,000	3.11	42.9
Lifted cavity transformer	202,822	30,000	3.15	55.7
FA-Gaussian (closed form)	–	–	–	38.3
NL-FA (closed form)	–	–	–	34.4

Training recipe (all synthetic-trained models except the transformer).

- Pool of 60,000 tasks (seed 5150).
- Minibatches of 128 tasks × 8 queries.
- AdamW with learning rate 1.8×10^{-3}, cosine decay to 10% and weight decay 10^{-4}.
- Gradient clipping at 5 and 20,000 steps.
- Seeds 1–3.

No hyperparameter search was run for any model.

Controls.

- **Residual MLP on the same anchor** (15,490 parameters). It receives the FA-Gaussian prediction, the same per-sensor support encodings (concatenated in sensor order), the readings and mask, and a 55-dimensional support summary. It outputs a residual correction to the FA-Gaussian mean and log-variance, and is defined only for five sensors.
- **Lifted sites without cavity input.** The same sites with the cavity statistics removed.
- **1-D sites** ($K = 0$). The same machinery, with sites over y only.
- **Repository cavity network.** The three-round scalar-cavity architecture of Appendix D, retrained with the shared recipe.

Transformer baseline (202,562 parameters). The transformer follows TabPFN v2 [15]:

- every cell is a token, embedded by a linear map of its value or by a learned missing-value token;
- sensors receive random per-task feature embeddings, so the model is width-agnostic and permutation equivariant in distribution;
- the target column has its own embedding, and query targets use a learned token;
- each of 4 blocks alternates attention across the columns of a row with attention across rows within a column (all rows attend to the support rows only), followed by an MLP ($d = 64$, 4 heads, hidden width 128, pre-LayerNorm);
- targets are standardized by the support mean and standard deviation, and the output is a Gaussian mean and log-variance.

It was trained on fresh tasks at every step (16 tasks $\times$ 16 queries, AdamW with learning rate 10^{-3}, 1,000 warm-up steps, cosine decay, clipping at 1, 30,000 steps), with the same query masks as the other models. A second copy (*all masks*) saw query masks with 0–4 missing sensors, drawn uniformly. Evaluation uses a fixed seed for the random feature embeddings.

Lifted cavity transformer (protocol v3, 202,822 parameters). The backbone is the transformer above, unchanged. Its output head is replaced by a zero-initialized linear map from each query cell's final token to four numbers $(\delta_j^{\mathrm{off}}, \delta_j^{\mathrm{slope}}, \delta_j^{D}, s_j)$, used exactly as in (4) with the linearization point $\bar{y}$ set to the anchor's closed-form posterior mean (one round, $K = 1$, gradients not propagated through $\bar{y}$). Readings of missing sensors are replaced by the missing-value token before the backbone and contribute no site, so they never enter the prediction (a unit test checks both this and the identity with the anchor's closed form at initialization). Anchors are computed on the fly for each training task. Training, masks, seed and evaluation are those of the transformer. Figure 3 compares the two training curves.

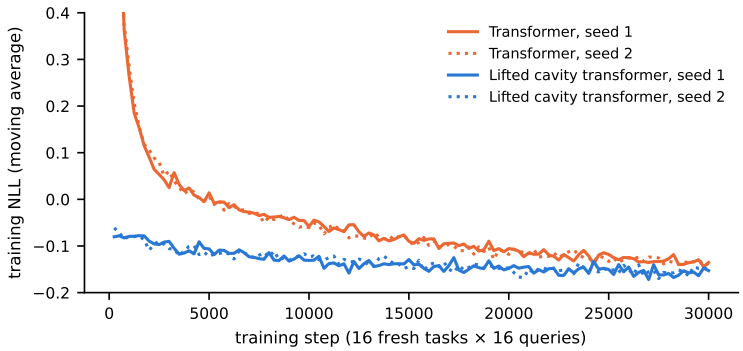


Figure 3: Training NLL (moving average; fresh tasks with at most one missing sensor) of the transformer and the lifted cavity transformer, which share backbone, recipe and seed. The lifted output layer starts at its anchor's closed form, so it begins far lower; the plain transformer must first learn fusion from scratch.

C Baselines and references

Support-only closed forms.

- **EM-Gaussian:** the conditional of y given x_S under the EM joint Gaussian (40 iterations).
- **FA-Gaussian:** the closed form of §4, with EM run for 40 iterations. The networks' anchors use 8 iterations and no variance inflation, so the reported FA-Gaussian is a slightly stronger baseline than the untrained networks (on F1 with two missing sensors, 0.177 against 0.187).
- **NL-FA:** per-sensor cubic response curves in the standardized target. The intercept and slope are unpenalized; the curvature terms are ridge-penalized ($\lambda = 100$) and clamped to the support's target range for linear extrapolation. The residual covariance has a one-factor

Table 5: Every declared endpoint of protocols v2–v4. Protocols v2/v3 preceded their panels; v4 was amended after partial data/reference inspection, before compared-model scoring (protocol v1: Appendix D). Gain in nats of NLL, positive when the first-named model is better; paired 95% intervals, cluster-robust over weeks on Beijing. v4 endpoints average three training seeds per task. Pass: gain ≥ 0.01 and lower bound > 0, except the non-inferiority endpoints E7b and E10 (lower bound > -0.02).

ID	Data	Comparison	Gain [95% CI]	Result
E1	F1, $k = 2$	lifted cavity network vs NL-FA	$+0.054$ [$+0.043$, $+0.065$]	pass
E2	F1, $k = 2$	lifted cavity network vs transformer	$+0.033$ [$+0.027$, $+0.040$]	pass
E3a	F3 (16 sensors), $k = 2$	lifted cavity network vs FA-Gaussian	$+0.148$ [$+0.115$, $+0.181$]	pass
E3b	F3 (16 sensors), $k = 2$	lifted cavity network vs transformer	$+0.144$ [$+0.127$, $+0.161$]	pass
E4a	Beijing $PM_{2.5}$	lifted cavity network (FT) vs BLR	$+0.194$ [$+0.131$, $+0.257$]	pass
E4b	Beijing $PM_{2.5}$	lifted cavity network (FT) vs transformer (FT)	-0.118 [-0.145, -0.091]	**fail**
E5	G1, $k = 2$	LCT vs transformer	$+0.035$ [$+0.027$, $+0.042$]	pass
E6	G3 (16 sensors), $k = 2$	LCT vs transformer	$+0.150$ [$+0.133$, $+0.167$]	pass
E7a	Beijing NO_2	LCT (FT) vs lifted cavity network (FT)	$+0.062$ [$+0.024$, $+0.100$]	pass
E7b	Beijing NO_2	LCT (FT) vs transformer (FT), non-inferiority	-0.021 [-0.093, $+0.052$]	**fail**
E8	H1, $k = 2$	LCT-L vs transformer-L	pending	–
E9	H3 (16 sensors), $k = 2$	LCT-L vs transformer-L	pending	–
E10	Beijing-new	LCT-L (FT) vs transformer-L (FT), non-inferiority	pending	–
E11	Beijing-new, $+6$ removed	LCT-L (FT) vs transformer-L (FT)	pending	–
E12	Beijing-new	LCT-L (FT) vs TabPFN v2 (in context)	pending	–

structure. Exact inference is done on a 401-point grid. The degree and λ were tuned on the development panel only: free degree selection by leave-one-out produced catastrophic extrapolation.

- **BLR:** evidence-maximized Bayesian linear regression [23] on complete-case rows, with predictive variance including parameter uncertainty.
- **Ridge:** complete-case ridge with leave-one-out residual variance, and the original mean-imputed ridge.
- **Gaussian process (post hoc):** for each episode and observed-sensor set, an ARD squared-exponential kernel plus white noise on the complete-case calibration rows, with hyperparameters by type-II maximum likelihood (one restart).
- **k-nearest neighbours (post hoc):** on the standardized observed sensors of complete-case calibration rows, with $k \in \{1, 2, 3, 5, 7, 10\}$ chosen per episode and sensor set by leave-one-out NLL on the support and the leave-one-out mean squared error as predictive variance.

EM-Gaussian, FA-Gaussian and ridge inflate their predictive variance by $n/(n - |S| - 1)$.

Sign-conditioned HMC reference. The reference uses HMC [27] over the 25 (or $5P$) task parameters. It uses the generator priors, transformed to unconstrained coordinates. The shared nuisance is marginalized analytically through a rank-one-plus-diagonal covariance, using only the observed entries of each calibration row. The sign of each a_j is fixed by the support correlation. The lower bound $|a_j| \geq 0.6$ does not guarantee correct sign identification from a finite noisy support. Consequently this reference omits posterior uncertainty over response signs and is not a certified Bayes-optimal ceiling.

The sampler runs 4 chains per task, each started from a MAP fit. Warm-up has two phases: step-size adaptation (target acceptance 0.75) and diagonal mass-matrix estimation from 150 iterations, followed by 150 more adaptation iterations. We keep 50 draws per chain, thinned by 4, with 12 leapfrog steps.

The predictive density of a query uses the Monte Carlo samples, including the query's own reweighting: $p(y \mid x_S, \mathcal{D}) = \sum_m p(y, x_S \mid \theta_m) / \sum_m p(x_S \mid \theta_m)$, with $p(x_S \mid \theta_m)$ by trapezoid quadrature on 301 points. On every panel where it was run, R-hat on identifiable parameters was below 1.1 for at least 99.9% of parameters. Run with the true parameters, the predictive formula reproduces the exact oracle to within 2×10^{-3} nats (a unit test).

D Protocols and audit trail

How the method arose. The lifted cavity network replaced an earlier design that failed. That design was a three-round EP-style network whose sites live on y alone and are updated from the scalar cavity of y. It was evaluated under pre-registered gates on the same synthetic family. Its first version collapsed to the prior during training. A repaired version, with sites initialized from support moments, trained successfully but was beaten by a ridge regression. On that panel the oracle, the ridge control and the best repaired seed scored:

Method	NLL, two sensors missing
Oracle	-0.046
Ridge control	0.396
Repaired scalar-cavity network (best seed)	0.403

Computing the oracle exposed three problems. First, the control was weak: it mean-imputed missing calibration entries, and a support-only EM or factor-analysis Gaussian scores 0.168 or 0.156. Second, scalar sites are structurally limited (§3). Third, the learners were starved: 512 reused tasks and 3,000 updates. Retraining the scalar-cavity architecture with our recipe (60,000 tasks, 20,000 steps) improves it by only about 0.05 nats, and on F1 it remains 0.159 nats worse than the untrained FA-Gaussian (§6.4).

Protocol v1. After development on panels that are now retired, five endpoints were committed to version control before their panels were generated:

- fresh synthetic tasks, against FA-Gaussian and against the residual MLP;
- 8-sensor transfer;
- stronger nonlinearity;
- a new real target (NO_2 on the Air Quality device).

All five passed (Table 6). Each endpoint was evaluated once, and no model was retrained after the commit.

Table 6: Protocol v1 endpoints (two missing sensors; positive gain favours the lifted cavity network; pass requires gain ≥ 0.01 and a positive lower bound).

Endpoint	Comparison	Gain [95% CI]	Result
E1	Fresh tasks, vs FA-Gaussian	$+0.074$ [$+0.064$, $+0.084$]	pass
E2	Fresh tasks, vs residual MLP	$+0.016$ [$+0.012$, $+0.020$]	pass
E3	8 sensors, vs FA-Gaussian	$+0.115$ [$+0.097$, $+0.132$]	pass
E4	Nonlinearity 0.8, vs residual MLP	$+0.028$ [$+0.020$, $+0.036$]	pass
R1	Air Quality NO_2, 23 later weeks, vs EM-Gaussian	$+0.112$ [$+0.050$, $+0.173$]	pass

Protocol v2. Protocol v2 was likewise committed before any of its panels or test splits existed. It adds:

- the transformer baseline, NL-FA, BLR and the sign-conditioned HMC reference;
- wider panels (8 and 16 sensors), and shifts in nonlinearity and support size;
- the Beijing benchmark, whose test period had never been scored.

NL-FA's settings and BLR's choice as the strongest real-data closed form were fixed on development data (Appendix C). The gas-sensor-drift dataset [34] was examined during development and then excluded, before any test split was built. Its heavy-tailed sensor outliers made the mean NLL of every Gaussian method dominated by a few episodes, with mean and median differing by more than 100 nats, so it cannot separate methods. One deviation was recorded before any Beijing score existed: some query hours have no other station observed, and complete-case BLR cannot fit zero features, so for an empty sensor set it returns the support mean with variance $s^2(1 + 1/n)$, like the other closed forms' prior predictive.

Protocol v3. After protocol v2's real-data endpoint failed, protocol v3 was committed before any of its panels or targets existed. It fixes one model (the lifted cavity transformer of §6.6, trained once with the transformer's recipe and seed and evaluated at its final checkpoint), two new synthetic panels (G1, G3), two Beijing pollutants never scored by any model (NO_2, primary; CO, descriptive), the fine-tuning recipe of protocol v2, and four endpoints (Table 5). The lifted cavity transformer's scores on the v2 panels are descriptive only (Appendix F). Two container restarts interrupted its training before any v3 score existed; it was retrained from scratch with the same seed, and later runs checkpoint every random state so that a resumed run is bitwise identical to an uninterrupted one (a unit test). The protocol document logs both events.

Protocol v4 status. The larger three-seed comparison and pretrained TabPFN evaluation are still in progress. No v4 efficacy endpoint is reported in this version.

Protocol v4. Protocol v4 was committed after protocol v3's results and before any of its panels or targets existed. It fixes:

- two models, each trained on a GPU with three seeds: transformer-L and LCT-L, with identical 2.1M-parameter backbones and fixed 40,000-step recipes;
- the pretrained TabPFN v2 regressor, used in context only, with eight estimators and a fixed random state;
- two fresh synthetic panels (H1, H3);
- three Beijing pollutants never scored by any model (PM_{10}, SO_2 and O_3, pooled as Beijing-new);
- the fine-tuning recipe of protocol v2;
- five endpoints (Table 5).

Amendment. One amendment was recorded before any compared model was trained or scored on a new target. Building the Beijing-new panels showed two things:

- The readings are integers, and 34.2% of SO_2 and 12.8% of O_3 test readings equal the floor value $2\,\mu\mathrm{g\,m^{-3}}$.
- Three SO_2 support sets were constant.

A continuous NLL can reward concentrating density on quantization values. Histogram and Gaussian outputs can benefit differently from this, complicating their comparison. We therefore dequantize [32]. Every integer reading of every station becomes reading $+\mathcal{U}(-\frac{1}{2}, \frac{1}{2})$ before the log. The noise is fixed per pollutant and shared by the test and fine-tuning episodes. Readings on finer grids, which occur only in 2013–2015, are kept as they are. Before the amendment, the only outputs seen were the closed-form reference scores printed while the undequantized panels were built; those panels were discarded.

GPU-side changes. Every change on the GPU side affects speed or robustness only:

- batch prefetching;
- longer checkpoint intervals, with a fallback to the previous checkpoint;
- a cached fine-tuning pool.

An interrupted 12-update LCT CPU run resumes to bitwise-identical weights in the unit test; this does not establish bitwise reproducibility of the full GPU runs or every execution optimization. TabPFN predicts all conditions of an episode in one call; its query rows do not interact. The third training seeds of the 203k transformers are post hoc.

Interpretation limits. The five decisions use nominal endpoint-wise 95% intervals, without a multiplicity correction or global superiority rule. Averaging three fitted seeds within each task conditions on those fits; the intervals omit training-randomness uncertainty. Week clustering does not establish independence between adjacent weeks. E12 compares target-source-fine-tuned LCT-L with in-context-only TabPFN: episode support is matched, but prior target-specific training is unequal, so this comparison cannot isolate the architecture. The real-data estimand is contemporaneous within-week adaptation, not forecasting.

Table 7: Generalization beyond the training distribution (fresh panels of 128 tasks). Mean test NLL with two missing query sensors. Every learned model was trained on $P{=}5$ sensors, nonlinearity 0.4 and $n{=}48$ support rows; the residual MLP is defined only for five sensors. Best non-privileged entry per column in bold; italic rows are controls or descriptive (the lifted cavity transformer was designed after protocol v2) and are not bolded.

Method	$P{=}8$	$P{=}16$	nonlin. 0.8	linear	$n{=}16$	$n{=}96$
Oracle	−0.598	−1.142	−0.086	−0.066	−0.060	−0.043
FA-Gaussian	−0.296	−0.797	0.334	**0.009**	0.510	0.114
EM-Gaussian	−0.160	2.977	0.311	0.034	1.140	0.112
NL-FA	−0.327	−0.841	0.258	0.030	0.496	0.073
BLR	0.615	0.058	0.331	0.061	3.690	0.125
Lifted cavity (ours)	**−0.427**	**−0.945**	**0.191**	0.031	**0.363**	**0.052**
Transformer	−0.362	−0.801	**0.191**	0.085	0.523	0.081
Residual MLP	–	–	0.235	0.036	0.371	0.074
Transformer, all masks	−0.343	−0.790	0.190	0.080	0.483	0.080
Lifted cavity transformer (v3)	−0.423	−0.949	0.179	0.029	0.457	0.047

Table 8: Ablations on F1 (mean test NLL). Same training recipe for every learned row.

Variant	$k{=}0$	$k{=}1$	$k{=}2$	$k{=}3$
Lifted cavity, $K = 1$ (ours)	−0.265	−0.112	0.097	0.395
Lifted cavity, $K = 2$	−0.267	−0.112	0.098	0.397
Lifted sites, no cavity input	−0.221	−0.074	0.123	0.399
Scalar sites ($K = 0$), same machinery	−0.070	0.061	0.232	0.473
Scalar-cavity network (retrained)	0.055	0.178	0.336	0.555
FA-Gaussian (untrained)	−0.152	−0.011	0.177	0.439

Execution audit and withheld decisions (October 11). The bounded GPU continuation completed and 24 learned checkpoints plus 35 score archives were collected. The strict independent verifier stopped before endpoint arithmetic: all 12 fine-tunes from seeds 2 and 3 lacked the required checkpoint-identity sidecar. The executed driver checked completion before fine-tuning but omitted the corresponding check after a fresh fit, then proceeded to scoring. Revisiting seed 1 created its six sidecars later; these were absent from the initial recovery snapshot. Therefore even their later presence does not establish pre-scoring attestation. This is a provenance deviation, not an efficacy result. We retain all artifacts and the failed gate, do not backfill historical identities, and report no verified v4 decisions, prediction replay or v4 reproduction claim. Entries marked "pending" in Table 5 mean unverified and withheld. The development driver now binds fresh fine-tunes before scoring and refuses existing unbound scores; four orchestration regression tests pass. That prospective repair does not repair the executed protocol's history.

Post-hoc analyses. Everything not named in a protocol is post hoc and labeled as such: the per-episode Gaussian process and k-nearest neighbours, the second training seeds of both transformers, the fine-tuning diagnostics of §6.5, and the exploratory runs of Appendix F.

E Broader impact

Calibrating low-cost environmental sensors widens access to air-quality monitoring. Overconfident predictions could mislead exposure decisions, which is why we report coverage alongside likelihood. We recommend validating calibration on each new deployment before relying on predicted intervals.

F Additional results

Table 9: Calibration and point accuracy. Coverage of the central 90% predictive interval (target 0.90), continuous ranked probability score (CRPS, lower is better) and RMSE. F1 with two missing sensors; Beijing test with natural missingness (fine-tuned learned models).

	F1, $k = 2$			Beijing, natural		
Method	Cov. 90	CRPS	RMSE	Cov. 90	CRPS	RMSE
FA-Gaussian	0.873	0.168	0.316	0.824	0.189	0.379
EM-Gaussian	0.860	0.170	0.318	0.807	0.207	0.423
BLR	0.866	0.174	0.327	0.861	0.182	0.374
Lifted cavity (ours)	0.901	0.160	0.305	0.914	0.171	0.357
Transformer	0.902	0.165	0.314	0.914	0.158	0.341
Residual MLP	0.902	0.162	0.309	–	–	–

Table 10: Post-hoc fine-tuning variants on Beijing $PM_{2.5}$ (test period of protocol v2; mean test NLL). Neither more nuisance dimensions nor longer fine-tuning closes the gap to the fine-tuned transformer.

Fine-tuned model	natural	$+3$	$+6$
Lifted cavity network, standard recipe (seed 1)	0.207	0.237	0.304
with $K = 2$	0.205	0.239	0.300
with $5\times$ fine-tuning steps	0.169	0.201	0.276
Transformer, standard recipe	0.087	0.121	0.236

Table 11: Second training seeds of the transformer and the lifted cavity transformer (post hoc). Endpoint comparisons recomputed with each seed and with both averaged; the lifted cavity network always averages its three seeds. Gains in nats, paired 95% intervals (cluster-robust over weeks on Beijing).

ID	Data	Comparison	Seed 1 (protocol)	Seed 2	Seeds averaged
E2	F1, $k = 2$	lifted vs transformer	$+0.033$ [$+0.027$, $+0.040$]	$+0.037$ [$+0.030$, $+0.044$]	$+0.035$ [$+0.029$, $+0.042$]
E3b	F3, $k = 2$	lifted vs transformer	$+0.144$ [$+0.127$, $+0.161$]	$+0.155$ [$+0.139$, $+0.171$]	$+0.149$ [$+0.134$, $+0.165$]
E5	G1, $k = 2$	LCT vs transformer	$+0.035$ [$+0.027$, $+0.042$]	$+0.037$ [$+0.028$, $+0.046$]	$+0.036$ [$+0.028$, $+0.044$]
E6	G3, $k = 2$	LCT vs transformer	$+0.150$ [$+0.133$, $+0.167$]	$+0.161$ [$+0.142$, $+0.180$]	$+0.155$ [$+0.139$, $+0.172$]
E4b	Beijing $PM_{2.5}$	lifted vs transformer (FT)	-0.118 [-0.145, -0.091]	-0.115 [-0.159, -0.070]	-0.116 [-0.150, -0.083]
E7a	Beijing NO_2	LCT vs lifted (FT)	$+0.062$ [$+0.024$, $+0.100$]	$+0.042$ [-0.023, $+0.108$]	$+0.052$ [$+0.002$, $+0.103$]
E7b	Beijing NO_2	LCT vs transformer (FT)	-0.021 [-0.093, $+0.052$]	-0.042 [-0.097, $+0.012$]	-0.032 [-0.093, $+0.030$]

Table 12: Panel F2 (8 sensors; 128 tasks): mean test NLL by number of missing sensors.

Method	k=0	k=1	k=2	k=3
Oracle	-0.807	-0.712	-0.598	-0.459
FA-Gaussian	-0.462	-0.387	-0.296	-0.183
EM-Gaussian	0.032	-0.133	-0.160	-0.120
NL-FA	-0.485	-0.414	-0.327	-0.215
BLR	0.187	0.398	0.615	0.028
Ridge (complete case)	0.064	-0.003	-0.033	-0.015
Lifted cavity (ours)	-0.615	-0.530	-0.427	-0.302
Lifted cavity, $K = 2$	-0.618	-0.531	-0.427	-0.302
Lifted sites, no cavity	-0.551	-0.469	-0.369	-0.250
Scalar sites ($K = 0$)	-0.295	-0.228	-0.148	-0.057
Transformer	-0.568	-0.474	-0.362	-0.229
Transformer, all masks	-0.545	-0.452	-0.343	-0.212
Lifted cavity transformer (v3)	-0.611	-0.526	-0.423	-0.299

Table 13: Panel F3 (16 sensors; 128 tasks): mean test NLL by number of missing sensors.

Method	k=0	k=1	k=2	k=3
Oracle	−1.223	−1.184	−1.142	−1.096
FA-Gaussian	−0.864	−0.832	−0.797	−0.758
EM-Gaussian	6.300	4.851	2.977	1.553
NL-FA	−0.910	−0.876	−0.841	−0.803
BLR	0.095	0.078	0.058	0.054
Ridge (complete case)	0.093	0.089	0.081	0.086
Lifted cavity (ours)	−1.018	−0.982	−0.945	−0.902
Lifted cavity, $K = 2$	−1.017	−0.981	−0.944	−0.901
Lifted sites, no cavity	−0.952	−0.916	−0.880	−0.836
Scalar sites ($K = 0$)	−0.658	−0.627	−0.591	−0.554
Transformer	−0.886	−0.846	−0.801	−0.747
Transformer, all masks	−0.869	−0.832	−0.790	−0.738
Lifted cavity transformer (v3)	−1.020	−0.985	−0.949	−0.905

Table 14: Panel F4 (nonlinearity 0.8; 128 tasks): mean test NLL by number of missing sensors.

Method	k=0	k=1	k=2	k=3
Oracle	−0.513	−0.331	−0.086	0.261
FA-Gaussian	0.050	0.170	0.334	0.568
EM-Gaussian	0.050	0.145	0.311	0.559
NL-FA	−0.032	0.089	0.258	0.507
BLR	0.208	0.188	0.331	0.568
Ridge (complete case)	0.138	0.208	0.338	0.571
Lifted cavity (ours)	−0.185	−0.026	0.191	0.498
Lifted cavity, $K = 2$	−0.200	−0.035	0.186	0.497
Lifted sites, no cavity	−0.068	0.075	0.261	0.519
Scalar sites ($K = 0$)	−0.013	0.130	0.314	0.570
Transformer	−0.226	−0.043	0.191	0.503
Transformer, all masks	−0.221	−0.038	0.190	0.486
Residual MLP	−0.107	0.039	0.235	0.508
Scalar-cavity network	0.133	0.256	0.416	0.635
Lifted cavity transformer (v3)	−0.196	−0.037	0.179	0.482

Table 15: Panel F5 (linear sensors; 128 tasks): mean test NLL by number of missing sensors.

Method	k=0	k=1	k=2	k=3
Oracle	−0.455	−0.288	−0.066	0.244
FA-Gaussian	−0.362	−0.203	0.009	0.305
EM-Gaussian	−0.244	−0.142	0.034	0.308
NL-FA	−0.337	−0.178	0.030	0.318
BLR	−0.135	−0.085	0.061	0.322
Ridge (complete case)	−0.169	−0.092	0.062	0.323
Lifted cavity (ours)	−0.323	−0.172	0.031	0.318
Lifted cavity, $K = 2$	−0.320	−0.170	0.033	0.318
Lifted sites, no cavity	−0.343	−0.189	0.018	0.309
Scalar sites ($K = 0$)	−0.085	0.031	0.186	0.415
Transformer	−0.278	−0.122	0.085	0.371
Transformer, all masks	−0.281	−0.123	0.080	0.353
Residual MLP	−0.304	−0.161	0.036	0.319
Scalar-cavity network	0.052	0.161	0.302	0.505
Lifted cavity transformer (v3)	−0.319	−0.171	0.029	0.315

Table 16: Panel F6 (16 support rows; 128 tasks): mean test NLL by number of missing sensors.

Method	k=0	k=1	k=2	k=3
Oracle	−0.446	−0.280	−0.060	0.252
FA-Gaussian	0.256	0.371	0.510	0.686
EM-Gaussian	24.900	4.682	1.140	0.739
NL-FA	0.314	0.388	0.496	0.661
BLR	0.898	0.857	3.690	0.887
Ridge (complete case)	0.317	0.391	0.500	0.624
Lifted cavity (ours)	0.073	0.195	0.363	0.604
Lifted cavity, $K = 2$	0.156	0.250	0.392	0.614
Lifted sites, no cavity	0.117	0.234	0.389	0.604
Scalar sites ($K = 0$)	0.204	0.293	0.428	0.644
Transformer	0.253	0.371	0.523	0.731
Transformer, all masks	0.256	0.359	0.483	0.653
Residual MLP	0.094	0.211	0.371	0.594
Scalar-cavity network	0.345	0.439	0.565	0.750
Lifted cavity transformer (v3)	0.237	0.333	0.457	0.643

Table 17: Panel F7 (96 support rows; 128 tasks): mean test NLL by number of missing sensors.

Method	k=0	k=1	k=2	k=3
Oracle	−0.429	−0.264	−0.043	0.269
FA-Gaussian	−0.232	−0.082	0.114	0.385
EM-Gaussian	−0.237	−0.086	0.112	0.385
NL-FA	−0.276	−0.127	0.073	0.353
BLR	−0.178	−0.056	0.125	0.390
Ridge (complete case)	−0.163	−0.047	0.129	0.391
Lifted cavity (ours)	−0.324	−0.163	0.052	0.353
Lifted cavity, $K = 2$	−0.320	−0.159	0.056	0.357
Lifted sites, no cavity	−0.275	−0.123	0.078	0.359
Scalar sites ($K = 0$)	−0.097	0.034	0.205	0.443
Transformer	−0.295	−0.133	0.081	0.379
Transformer, all masks	−0.297	−0.134	0.080	0.367
Residual MLP	−0.287	−0.131	0.074	0.360
Scalar-cavity network	0.047	0.161	0.309	0.516
Lifted cavity transformer (v3)	−0.329	−0.167	0.047	0.346

Table 18: Air Quality CO: mean test NLL for every number of simulated missing sensors (23 later weeks).

Method	k=0	k=1	k=2	k=3
EM-Gaussian	0.074	0.063	0.085	0.146
FA-Gaussian	0.052	0.086	0.116	0.154
NL-FA	0.144	0.120	0.091	0.124
Bayesian linear regression	1.037	0.433	0.216	0.202
Ridge (complete case)	0.824	0.337	0.208	0.210
GP, complete case†	3.364	1.056	0.503	0.146
k-nearest neighbours†	0.571	0.391	0.353	0.274
Lifted cavity, zero-shot	0.044	0.064	0.129	0.160
Transformer, zero-shot	−0.017	−0.006	0.081	0.154
Lifted cavity, fine-tuned (ours)	−0.040	−0.031	0.008	0.077
no synthetic pretraining	−0.001	−0.014	−0.006	0.077
no cavity input	−0.086	−0.065	−0.028	0.038
scalar sites ($K = 0$)	−0.026	−0.021	−0.003	0.075
Transformer, fine-tuned	−0.163	−0.149	−0.052	0.036
Residual MLP, fine-tuned	0.025	0.061	0.110	0.160
Lifted cavity transformer, fine-tuned‡	−0.099	−0.054	−0.027	0.044

Table 19: Air Quality NO_2: mean test NLL for every number of simulated missing sensors (23 later weeks).

Method	k=0	k=1	k=2	k=3
EM-Gaussian	−0.237	−0.174	−0.175	−0.168
FA-Gaussian	−0.196	−0.173	−0.155	−0.162
NL-FA	−0.090	−0.102	−0.142	−0.169
Bayesian linear regression	−0.055	−0.214	−0.178	−0.160
Ridge (complete case)	−0.200	−0.230	−0.176	−0.145
GP, complete case[†]	25.217	0.696	−0.198	−0.258
k-nearest neighbours[†]	−0.010	−0.020	−0.063	−0.128
Lifted cavity, zero-shot	−0.350	−0.295	−0.233	−0.176
Transformer, zero-shot	−0.360	−0.310	−0.231	−0.136
Lifted cavity, fine-tuned (ours)	−0.412	−0.366	−0.337	−0.273
no synthetic pretraining	−0.419	−0.365	−0.316	−0.248
no cavity input	−0.423	−0.375	−0.332	−0.275
scalar sites ($K = 0$)	−0.359	−0.337	−0.302	−0.231
Transformer, fine-tuned	−0.485	−0.429	−0.381	−0.294
Residual MLP, fine-tuned	−0.246	−0.198	−0.147	−0.106
Lifted cavity transformer, fine-tuned[‡]	−0.404	−0.396	−0.359	−0.292

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the scope: synthetic tasks from one generator family with exact oracle and sign-conditioned HMC references, and two real sensor networks; limitations are in Section 7.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 7 discusses the synthetic family, the real-data scope, Gaussian sites, the transformer compute budget and the untested pretrained tabular foundation models.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

Justification: Assumptions are stated in Propositions 1–4; complete proofs are in Appendix A.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Appendices B and C give architectures, recipes and baselines; seeds, panels and endpoints are fixed by three committed protocols (Appendix D).

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: Code, checkpoints, per-task scores and protocols are retained in the research repository, and public UCI data have recorded file hashes. An anonymized, independently tested supplementary archive is still pending.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 6 and Appendices B–C give data generation, training recipes, fine-tuning recipes and evaluation masks.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: All comparisons report paired 95% intervals over tasks or (for Beijing) cluster-robust intervals over weeks; learned models use three training seeds where stated.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)

- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: The reported v1–v3 efficacy experiments ran on two CPU cores; parameters, training time and inference cost are in Table 4 (Appendix B). Separate GPU engineering probes do not establish v4 efficacy; its full comparison is pending.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work uses public environmental sensor data and synthetic data and conforms to the Code of Ethics.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Appendix E discusses benefits for low-cost environmental monitoring and the risk of overconfident predictions; we report calibration.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The released models are small task-specific predictors with no foreseeable high risk of misuse.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: The UCI Air Quality and Beijing Multi-Site datasets are CC BY 4.0 and are cited; the TabPFN-v2-style baseline is our own implementation.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: Code and checkpoints are documented in the repository README and the protocols.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: No crowdsourcing or human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: No human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

Justification: An LLM-based coding assistant was used to implement experiments and draft text under the authors' direction; all numbers are generated by released scripts from saved scores.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.